

JARVIS

Linux Cluster Monitoring Agent Business Requirements Document

Prepared by Taiyaba Khatoon,

Business Systems Analyst

Jarvis Consulting

03/02/2026

Contents

1. Executive Summary	3
2. Business Problem	4
3. Project Objectives	4
4. Scope	4
5. Stakeholders.....	4
6. Business Requirements.....	5
6.1 Functional Requirements	5
6.2 Non-Functional Requirements	5
7. Implementation Overview	5
8. Schedule, Timeline, and Milestones.....	6
9. Cost-Benefit Analysis	6
10. Assumptions and Constraints	6
11. Risks and Mitigation	6
12. Acceptance Criteria	7
13. Glossary	7

1. Executive Summary

The Jarvis Linux Cluster Administration (LCA) team manages multiple Rocky Linux servers within a cluster environment. As the infrastructure grows, monitoring hardware configuration and system resource usage becomes increasingly complex. Currently, administrators must manually access individual servers to retrieve CPU, memory, and disk usage information. This manual approach is time-consuming, inconsistent, and does not support long-term trend analysis.

The purpose of this project is to design and implement a lightweight Linux Cluster Monitoring Agent that automates the collection of hardware and usage metrics. The solution will store collected data in a centralized PostgreSQL database, enabling structured SQL queries for reporting and analysis. By centralizing monitoring data, the LCA team will gain improved visibility into system performance and be able to make informed capacity planning decisions.

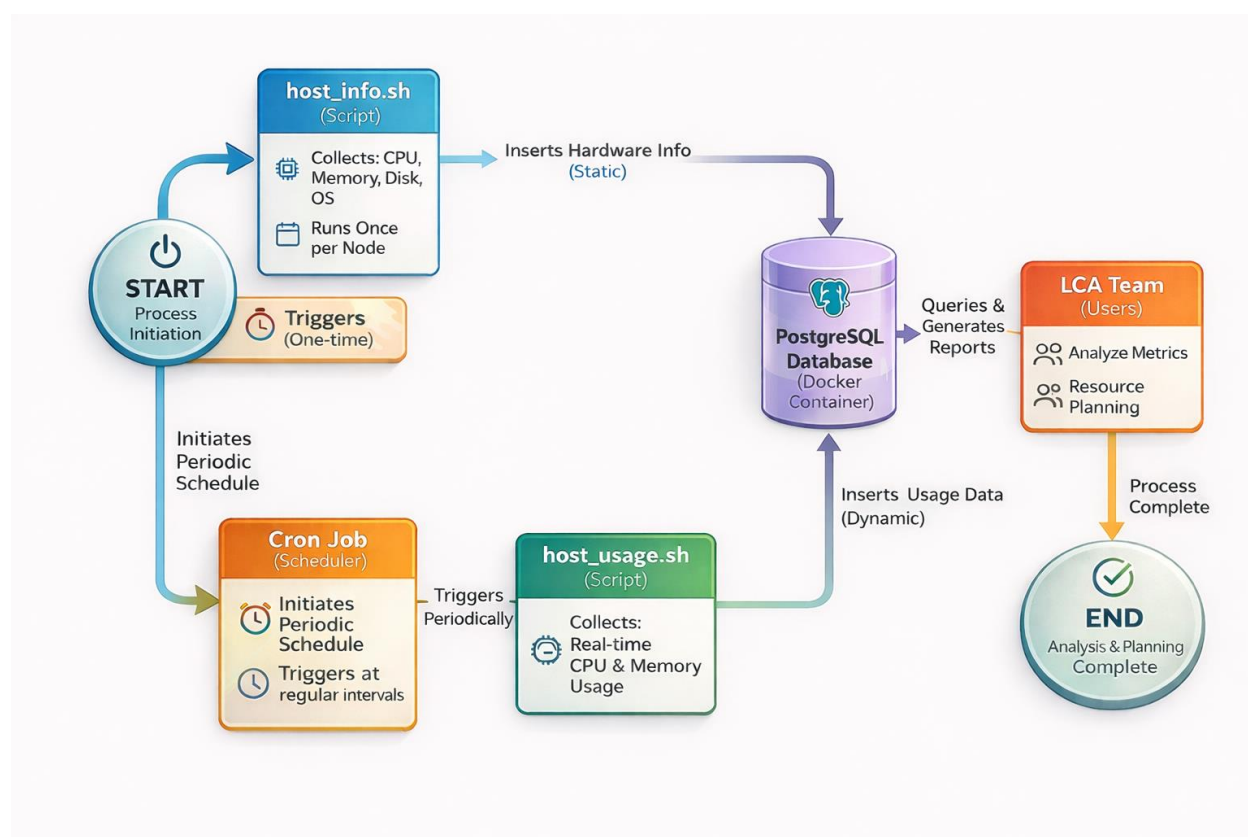


Figure 1: Overview Diagram

2. Business Problem

The existing monitoring process relies on manual data collection from each node. As the number of servers increases, this method does not scale effectively. Manual monitoring creates inconsistency in reporting and limits the ability to track historical resource utilization. Without centralized data storage, the team cannot easily identify usage patterns or anticipate when system resources may reach capacity.

This lack of structured monitoring introduces operational risk. Delayed visibility into performance metrics may result in over-provisioning, under-provisioning, or unexpected performance degradation. Therefore, a standardized and automated monitoring solution is required.

3. Project Objectives

The objective of this project is to implement a centralized monitoring mechanism capable of collecting both static hardware information and dynamic resource usage metrics from all nodes in the cluster. The system should store collected data in a relational database and support SQL-based retrieval for analysis. The solution must operate in an automated and repeatable manner, ensuring consistent metric collection without ongoing manual intervention. The final outcome should demonstrate a working Minimum Viable Product (MVP) that functions successfully within the existing cluster environment.

4. Scope

This project includes the development of Bash scripts to collect hardware specifications and ongoing usage metrics from Linux nodes. It also includes provisioning a PostgreSQL database using Docker to centrally store collected data. The monitoring scripts will be scheduled using cron to enable automated data capture at defined intervals. Basic SQL queries will be used to validate that data is successfully stored and retrievable.

The project does not include advanced alerting mechanisms, dashboard visualization tools, enterprise authentication integrations, or network-level configuration changes. These enhancements may be considered in future iterations once the MVP has been validated.

5. Stakeholders

The primary stakeholder for this project is the Linux Cluster Administration (LCA) team. This team is responsible for monitoring system health and making decisions related to resource allocation and capacity planning. The LCA team will define required metrics and validate that the collected data meets operational needs.

Supporting stakeholders include the development team responsible for implementing the monitoring scripts and database configuration, as well as the Database Administrator (DBA), who will maintain the PostgreSQL environment. Collaboration between these groups ensures successful deployment and ongoing reliability.

6. Business Requirements

6.1 Functional Requirements

Requirement ID	Requirement Description
FR-01	The system shall collect static hardware information from each Linux node and store it in the database.
FR-02	The system shall collect CPU and memory usage metrics at defined time intervals.
FR-03	All collected data shall be stored in a centralized PostgreSQL database.
FR-04	Monitoring scripts shall execute automatically using Cron scheduling.
FR-05	The database shall support SQL queries to retrieve metrics by host and time period.

6.2 Non-Functional Requirements

Requirement ID	Requirement Description
NFR-01	The solution shall support multiple nodes within the cluster environment.
NFR-02	Database credentials shall be handled securely.
NFR-03	The monitoring scripts shall be maintainable and clearly structured.
NFR-04	The system shall operate with minimal performance impact on nodes.

7. Implementation Overview

The implementation will begin with provisioning a PostgreSQL database using Docker. A database schema will be created to store static host information and dynamic usage metrics. The `host_info.sh` script will capture hardware details and insert them into the database. The `host_usage.sh` script will collect usage data and insert records at scheduled intervals. Cron configuration will automate recurring execution. Validation will be performed using SQL queries to confirm successful data insertion.

8. Schedule, Timeline, and Milestones

The project will be completed in phases including design, implementation, testing, and final demonstration. During the design phase, requirements will be reviewed and the database structure for storing host and usage data will be defined. Decisions about which metrics to collect and how often they should be captured will also be confirmed.

During implementation, the PostgreSQL database will be provisioned using Docker and the monitoring scripts will be developed. The `host_info.sh` script will collect static hardware details, and the `host_usage.sh` script will collect ongoing resource usage data.

In the testing phase, scripts will be executed manually to confirm that data is inserted correctly into the database. After successful validation, cron scheduling will be configured to automate recurring data collection. Logs and SQL queries will be used to verify that data is being captured consistently.

The main milestone of the project is delivering a working Minimum Viable Product (MVP) that can collect, store, and retrieve both host and usage metrics successfully.

9. Cost-Benefit Analysis

The primary costs associated with this project include development time for scripting, database setup, testing effort, and ongoing maintenance of scheduled tasks. However, the benefits include centralized data visibility, reduced manual monitoring effort, standardized metric collection, and improved capacity planning. The solution provides long-term operational efficiency and supports data-driven decision-making.

10. Assumptions and Constraints

This project assumes that all nodes are running Rocky Linux and that network connectivity to the centralized database is available. It also assumes that required Linux command-line tools are supported across environments and that cron scheduling is enabled.

A key constraint is that the project focuses only on an MVP implementation. The solution must be built using Linux CLI tools, Bash scripting, PostgreSQL, Docker, and SQL. Advanced monitoring features are outside the scope of this phase.

11. Risks and Mitigation

One potential risk is inconsistent metric collection across environments due to command output variations. This risk will be mitigated by standardizing CLI commands and validating outputs during testing. Another risk involves misconfigured Cron schedules, which could result in missing

data. This will be addressed by validating log outputs and database entries after deployment. Database availability is also a dependency, and connectivity checks will be included in deployment validation steps.

12. Acceptance Criteria

The project will be considered successful when the PostgreSQL database is provisioned correctly, the required tables are created, and both static and dynamic monitoring data are successfully inserted. Usage data must be captured automatically through scheduled execution, and SQL queries must retrieve stored metrics accurately. Documentation must clearly explain the business need, scope, and validation steps in a professional format.

13. Glossary

LCA – Linux Cluster Administration Team

Host – A Linux node within the cluster

Monitoring Agent – Bash scripts responsible for data collection

MVP – Minimum Viable Product